

MQTT

MQTT (originally an initialism of **MQ Telemetry Transport**^[a]) is a lightweight, publish-subscribe, machine to machine network protocol for message queue/message queuing service. It is designed for connections with remote locations that have devices with resource constraints or limited network bandwidth, such as in the Internet of Things (IoT). It must run over a transport protocol that provides ordered, lossless, bi-directional connections—typically, TCP/IP,^[1] but also possibly over QUIC.^[4] It is an open OASIS standard and an ISO recommendation (**ISO/IEC 20922**).

History

Andy Stanford-Clark (IBM) and Arlen Nipper (then working for Eurotech, Inc.) authored the first version of the protocol in 1999.^[5] It was used to monitor oil pipelines within the SCADA industrial control system.^[6] The goal was to have a protocol that is bandwidth-efficient, lightweight and uses little battery power, because the devices were connected via satellite link which, at that time, was extremely expensive.^[7]

Historically, the "MQ" in "MQTT" came from the IBM MQ (then 'MQSeries') product line, where it stands for "Message Queue". However, the protocol provides publish-and-subscribe messaging (no queues, in spite of the name).^[8] In the specification opened by IBM as version 3.1 the protocol was referred to as "MQ Telemetry Transport".^{[9][10]} Subsequent versions released by OASIS strictly refers to the protocol as just "MQTT", although the technical committee itself is named "OASIS Message Queuing Telemetry Transport Technical Committee".^[2] Since 2013, "MQTT" does not stand for anything.^{[11][8]}

In 2013, IBM submitted MQTT v3.1 to the OASIS specification body with a charter that ensured only minor changes to the specification could be accepted.^[2] After taking over maintenance of the standard from IBM, OASIS released version 3.1.1 on October 29, 2014.^{[12][13]} A more substantial upgrade to MQTT version 5, adding several new features,^[14] was released on March 7, 2019.^[1]

MQTT-SN (MQTT for Sensor Networks) is a variation of the main protocol aimed at battery-powered embedded devices on non-TCP/IP networks,^[15] such as Zigbee.^[16]

Overview

The MQTT protocol defines two types of network entities: a message broker and a number of clients. An MQTT broker is a server that receives all messages from the clients and then routes the messages to the appropriate destination clients.^[17] An MQTT client is any device (from a micro controller up to a fully-

MQTT



MQTT logo

Status	Published
Year started	1999
Latest version	5.0 ^[1] 7 March 2019
Organization	<u>OASIS</u>
Committee	OASIS Message Queuing Telemetry Transport Technical Committee ^[2]
Editors	Andrew Banks (IBM), Ed Briggs (Microsoft), Ken Borgendale (IBM), Rahul Gupta (IBM) ^[1]
Related standards	MQTT-SN ^[3]
Website	<u>mqtt.org</u> (<u>https://mqtt.org/</u>)

fledged server) that runs an MQTT library and connects to an MQTT broker over a network.^[18]

Information is organized in a hierarchy of topics. When a publisher has a new item of data to distribute, it sends a control message with the data to the connected broker. The broker then distributes the information to any clients that have subscribed to that topic. The publisher does not need to have any data on the number or locations of subscribers, and subscribers, in turn, do not have to be configured with any data about the publishers.

If a broker receives a message on a topic for which there are no current subscribers, the broker discards the message unless the publisher of the message designated the message as a retained message. A retained message is a normal MQTT message with the retained flag set to true. The broker stores the last retained message and the corresponding quality of service (QoS) for the selected topic. Each client that subscribes to a topic pattern that matches the topic of the retained message receives the retained message immediately after they subscribe. The broker stores only one retained message per topic.^[19] This allows new subscribers to a topic to receive the most current value rather than waiting for the next update from a publisher.

When a publishing client first connects to the broker, it can set up a default message to be sent to subscribers if the broker detects that the publishing client has unexpectedly disconnected from the broker.

Clients only interact with a broker, but a system may contain several broker servers that exchange data based on their current subscribers' topics.

A minimal MQTT control message can be as little as two bytes of data. A control message can carry nearly 256 megabytes of data if needed. There are fourteen defined message types used to connect and disconnect a client from a broker, to publish data, to acknowledge receipt of data, and to supervise the connection between client and server.

MQTT relies on the TCP protocol for data transmission. A variant, MQTT-SN, is used over other transports such as UDP or Bluetooth.

MQTT sends connection credentials in plain text format and does not include any measures for security or authentication. This can be provided by using TLS to encrypt and protect the transferred information against interception, modification or forgery.

The default unencrypted MQTT port is 1883. The encrypted port is 8883.^[20]

MQTT broker

The MQTT broker is a piece of software running on a computer (running on-premises or in the cloud), and could be self-built or hosted by a third party. It is available in both open source and proprietary implementations.

The broker acts as a post office. MQTT clients don't use a direct connection address of the intended recipient, but use the subject line called "Topic". Anyone who subscribes receives a copy of all messages for that topic. Multiple clients can subscribe to a topic from a single broker (one to many capability), and a single client can register subscriptions to topics with multiple brokers (many to one).

Each client can both produce and receive data by both publishing and subscribing, i.e. the devices can publish sensor data and still be able to receive the configuration information or control commands (MQTT is a bi-directional communication protocol). This helps in both sharing data, managing and controlling devices. A client cannot broadcast the same data to a range of topics, and must publish multiple messages to the broker, each with a single topic given.

With the MQTT broker architecture, the client devices and server application become decoupled. In this way, the clients are kept unaware of each other's information. MQTT, if configured to, can use TLS encryption with certificate, username and password protected connections. Optionally, the connection may require certification, in the form of a certificate file that a client provides and must match with the server's copy.

In case of failure, the broker software and clients can automatically hand over to a redundant/automatic backup broker. Backup brokers can also be set up to share the load of clients across multiple servers on site, in the cloud, or a combination of these.

The broker can support both standard MQTT and MQTT for compliant specifications such as Sparkplug.^[21] This can be done with same server, at the same time and with the same levels of security.

The broker keeps track of all the session's information as the device goes on and off, in a function called "persistent sessions". In this state, a broker will store both connection info for each client, topics each client has subscribed to, and any messages for a topic with a QoS of 1 or 2.^[22]

The main advantages of MQTT broker are:

1. Eliminates vulnerable and insecure client connections, if configured to
2. Can easily scale from a single device to thousands
3. Manages and tracks all client connection states, including security credentials and certificates, if configured to
4. Reduced network strain without compromising the security, if configured to (cellular or satellite network)

Message types

Connect

Waits for a connection to be established with the server and creates a link between the nodes.

Disconnect

Waits for the MQTT client to finish any work it must do, and for the TCP/IP session to disconnect.

Publish

Returns immediately to the application thread after passing the request to the MQTT client.

Version 5.0

In 2019, OASIS released the official MQTT 5.0 standard.^[1] Version 5.0 includes the following major new features:^[23]

- Reason codes: Acknowledgements now support return codes, which provide a reason for a failure.

- Shared subscriptions: Allow the load to be balanced across clients and thus reduce the risk of load problems
- Message expiry: Messages can include an expiry date and are deleted if they are not delivered within this time period.
- Topic alias: The name of a topic can be replaced with a single number

MQTT 5.0 also supports MQTT connections over the QUIC transport protocol. MQTT over QUIC offers improved performance by reducing the number of exchanges during the connection process, reducing overall latency, and offering better handling of network congestion and switching.^[24]

Quality of service

Each connection to the broker can specify a QoS measure.^[25] These are classified in increasing order of overhead:

- At most once – the message is sent only once and the client and broker take no additional steps to acknowledge delivery (fire and forget).
- At least once – the message is re-tried by the sender multiple times until acknowledgement is received (acknowledged delivery).
- Exactly once – the sender and receiver engage in a two-level handshake to ensure only one copy of the message is received (assured delivery).

This field does not affect handling of the underlying TCP data transmissions; it is only used between MQTT senders and receivers.

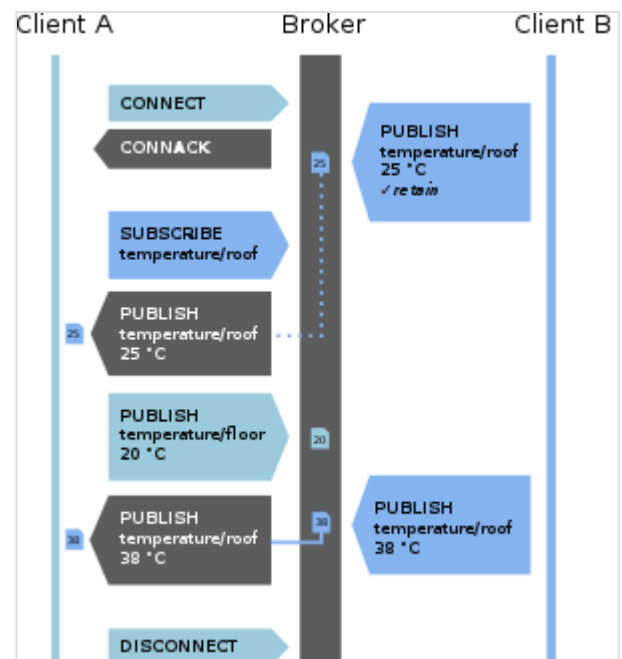
Security

Security of the MQTT protocol was compromised^[26] in 2020 by Italian researchers, executing Slow DoS Attacks on such protocol (see CVE-2020-13849 (<https://cve.mitre.org/cgi-bin/cvename.cgi?name=2020-13849>)).

Clustering

MQTT clustering is a technique employed to ensure high availability, fault tolerance, and scalability in MQTT deployments.^[27] As an efficient and lightweight messaging protocol, MQTT clustering allows for the creation of a resilient network of interconnected broker nodes, ensuring continuous and reliable message delivery even in the face of hardware failures or network disruptions.

EMQX's Implementation of Clustering



Example of an MQTT connection (QoS 0) with connect, publish/subscribe, and disconnect. The first message from client B is stored due to the retain flag.

EMQX is written in Erlang. Before version 5.0, in a cluster of EMQX Community's MQTT brokers,^[28] each broker node is mutually connected. For each node, there is a Mnesia^[29] database, a built-in database for Erlang, to store important data. One of the most important pieces of data stored in the database is the mapping from the topics to the IP addresses of the subscribers. Such mapping is called the subscription table. Another table called the routing table contains the mapping from topics to the IP addresses of the node managing these topics. Both tables guarantees that every message sent to the cluster will be forwarded to the correct subscriber. When a message with a specific topic is sent to a random node in the cluster, the node is going to check the routing table to see which node manages this topic. It will then forward the message to the managing node. After the managing node receives the message, it will check the subscription table to see which client subscribed this topic and send the message to that client.

However, there are some limitations of the Mnesia database:^[30] it is vulnerable to network issues, the result of which is split brain. Moreover, the size limitation of the cluster is seven nodes, which cannot ensure scalability. To deal with these issues, in the current version 5, Mnesia is enhanced to Mria. Mria is not a completely different database. Instead, it derives from Mnesia with some improvements. In addition to the database, the role of nodes in the cluster is split into two: core and replica. Core nodes are still connected to each other, but each core node is responsible for several replica nodes. Each replica node owns a complete replication of data from its core node and only connect to it. In the cluster, only core nodes are allowed both to write data into the database and to read data from it. In contrast, replica nodes are only allowed to read data. This architecture not only decreases the risk of split-brain situations by using Mria database but also improve the cluster's scalability by setting the replica nodes to relieve the pressure of the core nodes.

EMQX Enterprise, the paid version of EMQX Community, has a slightly different architecture of the clusters. One of the key differences is that instead of Mria, to achieve better data persistence, EMQX Enterprise makes use of RocksDB as its database.^[31]

See also

- Comparison of MQTT implementations
- Advanced Message Queuing Protocol (AMQP)
- Streaming Text Oriented Messaging Protocol (STOMP)
- Constrained Application Protocol (CoAP)
- Apache ActiveMQ
- Solace PubSub+^{[32][33]}
- RabbitMQ

Notes

- a. MQ stands for "message queues", as derived from the IBM MQ product name.

References

1. "MQTT Version 5.0" (<https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>). OASIS. 2019-03-07. Retrieved 2020-12-15.
2. "OASIS Message Queuing Telemetry Transport (MQTT) Technical Committee Charter" (<https://www.oasis-open.org/committees/mqtt/charter.php>). OASIS. Retrieved 2020-12-15.
3. "MQTT SN Subcommittee" (https://www.oasis-open.org/committees/tc_home.php?wg_abbrev=mqtt-sn). OASIS. Retrieved 2020-12-15.

4. "MQTT over QUIC: Next-Generation IoT Standard Protocol" (<https://www.emqx.com/en/blog/mqtt-over-quic#quic-vs-tcp-tls/>). EMQ. 2022-08-24.
5. "10th birthday party" (<https://web.archive.org/web/20150315025826/https://mqtt.org/2009/07/10th-birthday-party>). *MQTT.org*. July 2009. Archived from the original (<http://mqtt.org/2009/07/10th-birthday-party>) on March 15, 2015. Retrieved April 25, 2015.
6. "Transcript of IBM podcast" (https://www.ibm.com/podcasts/software/websphere/connectivity/piper_diaz_nipper_mq_tt_11182011.pdf) (PDF). *IBM.com*. November 2011. Retrieved January 7, 2021.
7. "Getting Started with MQTT" (<https://www.hivemq.com/blog/how-to-get-started-with-mqtt/>). HiveMQ. 2020-04-24.
8. Team, The HiveMQ. "Introducing the MQTT Protocol - MQTT Essentials: Part 1" (<https://www.hivemq.com/blog/mqtt-essentials-part-1-introducing-mqtt/>). *www.hivemq.com*. Retrieved 2021-09-26.
9. "MQTT v3.1 and MQTT v3.1.1 Differences" (https://www.oasis-open.org/committees/document.php?document_id=55095&wg_abbrev=mqtt). OASIS Message Queuing Telemetry Transport (MQTT) TC. 12 February 2015. Retrieved 19 August 2021.
10. "MQTT V3.1 Protocol Specification" (<https://public.dhe.ibm.com/software/dw/webservices/ws-mqtt/mqtt-v3r1.html>). Eurotech, International Business Machines Corporation (IBM). 2010. Retrieved 2020-12-15.
11. "OASIS MQTT Technical Committee Minutes of for the meeting of Thursday, 25th April 2013 Teleconference" (https://www.oasis-open.org/committees/download.php/49028/OASIS_MQTT_TC_minutes_25042013.pdf) (PDF).
12. "MQTT Version 3.1.1" (<http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>). 2014-10-29. Retrieved 2020-12-16.
13. "6 facts why it's worth upgrading to the brand new MQTT 3.1.1 version" (<https://www.hivemq.com/blog/6-facts-why-its-worth-upgrading-to-mqtt-3-1-1/>). 2014-10-30. Retrieved 2020-12-16.
14. "Differences between 3.1.1 and 5.0" (<https://github.com/mqtt/mqtt.org/wiki/Differences-between-3.1.1-and-5.0>). *GitHub*.
15. Stanford-Clark, Andy; Hong Linh Truong (November 14, 2013). "MQTT For Sensor Networks (MQTT-SN) Protocol Specification Version 1.2" (https://www.oasis-open.org/committees/download.php/66091/MQTT-SN_spec_v1.2.pdf) (PDF). *oasis-open.org*. OASIS Message Queuing Telemetry Transport (MQTT) Technical Committee. p. 28. Retrieved 2020-12-15.
16. "Introduction to MQTT-SN (MQTT for Sensor Networks)" (<http://www.steves-internet-guide.com/mqtt-sn/>). 25 January 2017. Retrieved 2020-09-16.
17. Yuan, Michael. "Getting to know MQTT" (<https://developer.ibm.com/articles/iot-mqtt-why-good-for-iot/>). *IBM Developer*. Retrieved 13 October 2019.
18. "Client, Broker / Server and Connection Establishment - MQTT Essentials: Part 3" (<https://www.hivemq.com/blog/mqtt-essentials-part-3-client-broker-connection-establishment/>). *hivemq.com*. Retrieved 13 October 2019.
19. "Retained Messages - MQTT Essentials: Part 8" (<https://www.hivemq.com/blog/mqtt-essentials-part-8-retained-messages/>). *hivemq.com*. Retrieved 13 October 2019.
20. "FAQ - Frequently Asked Questions | MQTT" (<http://mqtt.org/faq>). Retrieved 2020-03-19.
21. "MQTT Sparkplug/Tahu" (<https://www.cirrus-link.com/mqtt-sparkplug-tahu/>). *www.cirrus-link.com*. Retrieved November 5, 2019.
22. Cope, Stephen (2020). *MQTT For Complete Beginners*. p. 17. ISBN 9798779030762.
23. "What is MQTT? Definition and Details" (<https://www.paessler.com/it-explained/mqtt>). *www.paessler.com*. Retrieved 2020-06-09.

24. "QUIC Protocol: the Features, Use Cases and Impact for IoT/IoV" (<https://www.emqx.com/en/blog/quic-protocol-the-features-use-cases-and-impact-for-iot-iov>). *www.emqx.com*. Retrieved 2023-06-13.
25. "IBM Knowledge Center - IBM MQ - Using MQTT with IBM Integration Bus - Quality of service and connection management" (https://www.ibm.com/support/knowledgecenter/SSM KHH_10.0.0/com.ibm.etools.mft.doc/bc62020_.htm). *www.ibm.com*. Retrieved 2018-01-30.
26. Vaccari, I., Aiello, M., & Cambiaso, E. (2020). SlowITe, a novel denial of service attack affecting MQTT. *Sensors*, 20(10), 2932.
27. "High availability MQTT Cluster - Bevywise Networks" (<https://www.bevywise.com/blog/high-availability-mqtt-cluster/>). *www.bevywise.com*. Retrieved 2023-12-22.
28. "EMQX Docs - EMQX Clustering" (<https://www.emqx.io/docs/en/latest/deploy/cluster/introduction.html>).
29. "Yaws: Yet Another Web Server - Vinoski, Steve" (<https://www.computer.org/csdl/magazine/ic/2011/04/mic2011040090/13rUWjXZOB>).
30. "EMQX Docs - Routing Table Replication" (<https://www.emqx.io/docs/en/latest/design/cluster ing.html#routing-table-replication>).
31. EMQX Team. "Highly Reliable MQTT Data Persistence Based on RocksDB" (<https://www.emqx.com/en/blog/mqtt-persistence-based-on-rocksdb>). *emqx.com*. Retrieved 2024-03-01.
32. "APIs & Protocols" (<https://solace.com/products/apis-protocols/>). *Solace*. Retrieved 2021-04-08.
33. "MQTT 5.0 Support 🎉" (<https://solace.community/discussion/540/mqtt-5-0-support>). *Solace Community*. 4 January 2021. Retrieved 2021-04-08.

External links

- Official website (<https://mqtt.org/>) 
- Official website of Technical Committee (https://www.oasis-open.org/committees/tc_home.php?wg_abbrev=mqtt)
- MQTT Specifications
 - Version 5.0, 2019-03-07: PDF edition (<https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.pdf>), HTML edition (<https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>)
 - Version 3.1.1 Plus Errata 01, 2015-12-10: PDF edition (<https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.pdf>), HTML edition (<https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>)
 - Version 3.1.1, 2014-10-29: PDF edition (<https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.pdf>), HTML edition (<https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>)
 - Version 3.1, 2010: PDF edition (http://public.dhe.ibm.com/software/dw/webservices/ws-mqtt/MQTT_V3.1_Protocol_Specific.pdf) Archived (https://web.archive.org/web/20221204093708/http://public.dhe.ibm.com/software/dw/webservices/ws-mqtt/MQTT_V3.1_Protocol_Specific.pdf) 2022-12-04 at the Wayback Machine, HTML edition (<https://public.dhe.ibm.com/software/dw/webservices/ws-mqtt/mqtt-v3r1.html>)
- MQTT-SN Specifications
 - Version 1.2, 2013-11-14: PDF edition (https://www.oasis-open.org/committees/download.php/66091/MQTT-SN_spec_v1.2.pdf)

